SOFTWARE REQUIREMENTS
SPECIFICATION

VERITY

Fake News Detection Using BERT and Graph Convolutional Network

Prepared by Team-17
Under the Guidance of Ms. T. N. SriRanjani

19 September 2026

Academic Project — Version 1.0

A requirements specification for the VERITY academic prototype.

Document Control

Field	Value
Project	VERITY — Fake News Detection Using BERT and Graph Convolutional Network
Document	Software Requirements Specification (SRS)
Version	1.0
Prepared by	Team-17
Guidance	Ms. T. N. SriRanjani
Date	19 September 2026
Status	Academic project specification

Revision History

Version	Date	Reason for Changes	Author
1.0	19-Sep-2026	Initial consolidated SRS for the VERITY prototype.	Team-17

Table of Contents

Document Control 2

Table of Contents 3

1. Introduction 4

2. Overall Description 6

3. External Interface Requirements 9

4. System Features 10

5. Other Nonfunctional Requirements 14

6. Other Requirements 16

Appendix A: Glossary 17

Appendix B: Analysis Models 18

Appendix C: To Be Determined List 19

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of VERITY, a web-based fake news detection prototype that uses Natural Language Processing (NLP), BERT-based text representations, semantic similarity, and a Graph Convolutional Network (GCN) to classify news articles as REAL or FAKE.

Version 1.0 covers news input, text representation, semantic similarity, graph construction, GCN classification, prediction probabilities, related-article display, pipeline explanation, and the web interface. The system is intended for academic and research demonstration and is not intended to replace professional journalism, fact-checking organizations, or independent human verification.

1.2 Document Conventions

- **BERT** — Bidirectional Encoder Representations from Transformers.
- **GCN** — Graph Convolutional Network.
- **GNN** — Graph Neural Network.
- **NLP** — Natural Language Processing.
- **REAL** — News classified as belonging to the real-news class.
- **FAKE** — News classified as belonging to the fake-news class.
- **REQ-X** — Unique identifier for a requirement.
- **High / Medium / Low** — Requirement priority.
- The word **"shall"** indicates mandatory behavior.

1.3 Intended Audience and Reading Suggestions

- **Developers:** understand functionality, technologies, interfaces, constraints, and implementation requirements.
- **Project Team:** understand the complete workflow and module responsibilities.
- **Project Guide / Faculty:** review objectives, architecture, functionality, and expected behavior.
- **Testers:** derive test cases from functional and non-functional requirements.
- **End Users:** understand submission, prediction, and related information.

Suggested reading order: Introduction → Overall Description → External Interface Requirements → System Features → Nonfunctional Requirements → Other Requirements → Appendices.

1.4 Product Scope

VERITY is an AI-based web application designed to assist users in identifying potentially fake or misleading news articles. The system accepts a news headline or article and analyzes it using transformer-based language representation and graph-based learning.

- Automate preliminary fake-news classification.
- Extract contextual linguistic features from news content.
- Identify semantic relationships between articles.
- Use relationships between related articles during graph-based classification.
- Provide an understandable model prediction.
- Display predicted class probabilities.

- Show semantically related articles used in graph construction.
- Provide an intuitive interface for demonstrating the AI pipeline.

The output is a machine-learning classification and should be treated as an assistive signal rather than definitive proof of factual truth.

1.5 References

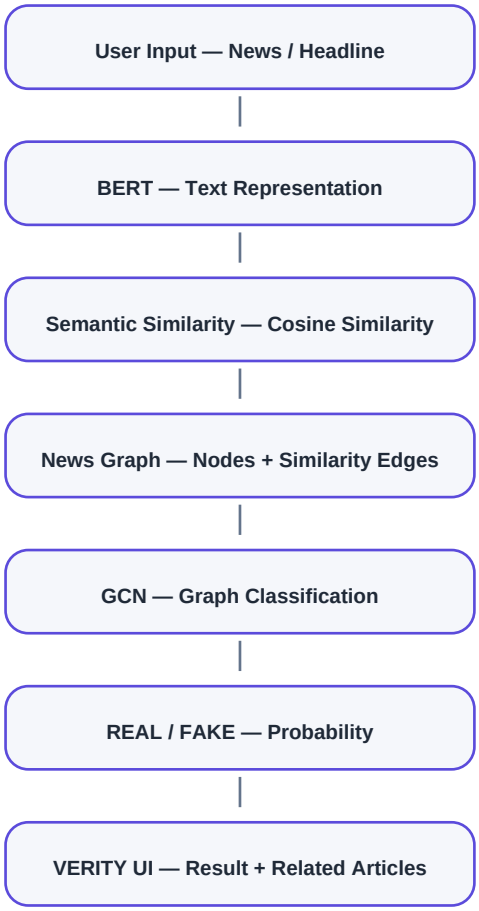
- Devlin, J., Chang, M. W., Lee, K., and Toutanova, K. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” NAACL-HLT, 2019.
- Kipf, T. N. and Welling, M. “Semi-Supervised Classification with Graph Convolutional Networks.” ICLR, 2017.
- Hugging Face Transformers documentation — tokenizer and pretrained BERT implementation.
- PyTorch documentation — tensor, neural-network, and model execution.
- PyTorch Geometric documentation — graph representation and GCN implementation.
- Streamlit documentation — web-based user interface.
- Scikit-learn documentation — cosine similarity and machine-learning utilities.

2. Overall Description

2.1 Product Perspective

VERITY is a self-contained academic prototype combining transformer-based NLP with graph-based deep learning. A submitted article is converted into a contextual embedding using BERT, compared with existing article embeddings, connected to semantically similar articles, and processed by a trained GCN.

High-level architecture



2.2 Product Functions

- Accept news text and validate submitted input.
- Tokenize input using the BERT tokenizer.
- Generate a 768-dimensional contextual representation.
- Compare the representation with existing article representations.
- Identify the five most semantically similar articles.
- Add relationships between the submitted article and selected similar articles.
- Apply the trained GCN to the resulting graph.
- Predict REAL or FAKE.
- Calculate and display class probabilities.
- Display related articles and similarity values.
- Explain the major analysis stages and show the system architecture.

- Handle invalid or empty input gracefully.

2.3 User Classes and Characteristics

2.3.1 General User

- Basic computer and web-browsing knowledge.
- Can enter or paste a news article or headline.
- Can submit the article for analysis.
- Can view prediction, probabilities, and related articles.
- Does not require programming or machine-learning knowledge.

2.3.2 Project Reviewer / Faculty

- May evaluate architecture, BERT integration, graph construction, GCN implementation, prediction process, interface, and overall functionality.

2.3.3 Developer / Project Team

- Requires knowledge of Python, machine learning, NLP, PyTorch, PyTorch Geometric, BERT/Transformers, and Streamlit.

2.4 Operating Environment

Category	Requirement
Processor	Intel/AMD x64 processor or equivalent.
RAM	Minimum 8 GB; 16 GB recommended.
Storage	Sufficient storage for dataset, pretrained BERT model, graph, and GCN model.
GPU	Optional; CUDA-compatible GPU may accelerate training and inference.
Operating System	Windows 10/11, Linux, or macOS.
Python	3.9 or later.
Browser	Chrome, Edge, Firefox, or equivalent modern browser.
Libraries	Streamlit, PyTorch, PyTorch Geometric, Hugging Face Transformers, Pandas, Scikit-learn.

2.5 Design and Implementation Constraints

- The system shall be implemented primarily using Python.
- BERT shall be used for contextual text representation.
- When bert-base-uncased is used, the representation is 768-dimensional.
- A Graph Convolutional Network shall be used for graph-based classification.
- PyTorch and PyTorch Geometric shall be used for deep-learning implementation.
- The prepared news dataset shall be used for training and evaluation.
- The web interface shall be implemented using Streamlit.
- The system shall operate within available computational resources.
- Model files shall be stored locally for the prototype.
- The system shall not represent a model prediction as independently verified factual truth.

2.6 User Documentation

- Installation and setup instructions.

- Project execution instructions.
- Instructions for submitting news.
- Description of prediction output.
- System architecture and technology description.
- Troubleshooting instructions.
- Project SRS document.

2.7 Assumptions and Dependencies

- Input contains meaningful news-related text.
- The pretrained BERT model, trained GCN model, processed graph, and labeled dataset are available.
- The input language is compatible with the selected BERT model.
- Required Python libraries are installed correctly.
- The environment has sufficient memory for models and graph data.
- Semantic similarity provides useful information for graph construction.
- The trained model generalizes reasonably to unseen articles.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide a web-based graphical user interface containing the following areas:

- **Header:** application name, system status, and AI-technology identification.
- **Article Input:** text area, placeholder guidance, Analyze button, Clear button, and optional sample input.
- **Prediction Output:** predicted class, REAL/FAKE label, REAL probability, FAKE probability, confidence/probability indicator, and explanation.
- **Analysis Pipeline:** BERT representation, semantic graph construction, and GCN classification.
- **Related Articles:** up to five semantically similar articles with similarity values.
- **Architecture:** NEWS ARTICLE → BERT → 768-D EMBEDDING → SEMANTIC GRAPH → GCN → REAL/FAKE.

The interface shall be responsive and usable on standard desktop browsers.

3.2 Hardware Interfaces

- CPU for computation.
- RAM for loading datasets and models.
- Optional GPU for accelerated computation.
- Local storage for datasets and model files.
- Display and input devices through the web browser.

No external sensors or specialized hardware devices are required.

3.3 Software Interfaces

Software	Interface / Responsibility
BERT / Transformers	Load tokenizer and pretrained model; tokenize news and generate contextual embeddings.
PyTorch	Tensor operations, neural-network execution, model loading, and CPU/GPU computation.
PyTorch Geometric	Graph representation, GCNConv layers, and graph neural-network computation.
Scikit-learn	Cosine similarity for finding semantically similar articles.
Pandas	Load processed CSV data and retrieve article content.
Streamlit	Web interface, user input, controls, prediction display, and UI components.

3.4 Communications Interfaces

- The prototype primarily operates through local communication between the Streamlit application and Python runtime.
- The Streamlit application communicates with the user's browser using HTTP.
- No external API communication is required for core prediction functionality.
- Continuous internet connectivity is not required after required models and datasets have been downloaded locally.

4. System Features

4.1 News Article Input

Priority: High.

Stimulus / Response Sequence

1. User opens VERITY.
2. System displays the input interface.
3. User enters or pastes news content.
4. User selects Analyze article.
5. System validates the input.
6. If valid, analysis begins; if empty, an input warning is displayed.

Functional Requirements

ID	Requirement	Priority
REQ-1	The system shall provide a text input area for news content.	High
REQ-2	The system shall accept a headline or full article.	High
REQ-3	The system shall reject empty input.	High
REQ-4	The system shall display an appropriate warning when empty input is submitted.	High
REQ-5	The system shall provide an Analyze button.	High
REQ-6	The system shall provide a mechanism to clear entered text.	Medium

4.2 BERT-Based Text Representation

Priority: High.

Stimulus / Response Sequence

1. User submits an article.
2. System tokenizes the article.
3. Tokenized input is passed to BERT.
4. BERT generates contextual representations.
5. System extracts the representation used by the classifier.
6. Representation is forwarded to graph processing.

Functional Requirements

ID	Requirement	Priority
REQ-7	The system shall tokenize submitted news using the BERT tokenizer.	High
REQ-8	The system shall use the bert-base-uncased model for text representation.	High
REQ-9	The system shall generate a 768-dimensional representation for the submitted article.	High
REQ-10	The system shall truncate excessively long input according to the configured maximum sequence length.	Medium
REQ-11	The system shall generate the BERT representation without modifying the pretrained model during inference.	High

4.3 Semantic Similarity and Graph Construction

Priority: High.

Stimulus / Response Sequence

1. System receives the BERT representation.
2. System compares it with existing article representations.
3. Cosine similarity is calculated.
4. The five most similar articles are identified.
5. A new node representing the submitted article is created.
6. Edges are created between the new node and the selected similar nodes.
7. The resulting graph is provided to the GCN.

Functional Requirements

ID	Requirement	Priority
REQ-12	The system shall calculate semantic similarity between the input article and existing article representations.	High
REQ-13	The system shall use cosine similarity for similarity calculation.	High
REQ-14	The system shall identify the five highest-similarity articles.	High
REQ-15	The system shall represent the submitted article as a new graph node.	High
REQ-16	The system shall create graph connections between the new article and its selected similar articles.	High
REQ-17	The system shall preserve the existing graph structure during inference.	High

4.4 GCN-Based Classification

Priority: High.

Stimulus / Response Sequence

1. The constructed graph is provided to the GCN.
2. The GCN processes node features and graph relationships.
3. Information is propagated through graph convolution layers.
4. The output layer generates class scores.
5. Softmax converts scores into class probabilities.
6. The class with the highest probability is selected.

Functional Requirements

ID	Requirement	Priority
REQ-18	The system shall use a trained GCN model for classification.	High
REQ-19	The GCN shall use the configured graph node features and graph edges.	High
REQ-20	The GCN shall contain graph convolution layers configured for the 768-dimensional BERT representation.	High
REQ-21	The system shall produce two classification outputs corresponding to REAL and FAKE.	High
REQ-22	The system shall apply softmax to obtain class probabilities.	High
REQ-23	The system shall select the class having the highest predicted probability.	High

4.5 Prediction Display

Priority: High.

Stimulus / Response Sequence

1. GCN generates the prediction.
2. System determines the predicted class.
3. System obtains class probabilities.
4. System displays the result and probability information.

Functional Requirements

ID	Requirement	Priority
REQ-24	The system shall display either REAL or FAKE as the predicted class.	High
REQ-25	The system shall display the predicted probability for the REAL class.	High
REQ-26	The system shall display the predicted probability for the FAKE class.	High
REQ-27	The system shall visually distinguish the prediction result.	Medium
REQ-28	The system shall indicate that the output is a model prediction rather than independently verified factual truth.	High

4.6 Related Article Display

Priority: Medium.

Stimulus / Response Sequence

1. Similarity scores are calculated.
2. The five highest-scoring articles are selected.
3. Their content is retrieved from the processed dataset.
4. Articles are displayed with similarity scores.

Functional Requirements

ID	Requirement	Priority
REQ-29	The system shall display up to five related articles.	Medium
REQ-30	The system shall display the semantic similarity value for each related article.	Medium
REQ-31	The system shall rank related articles according to similarity.	Medium

4.7 Analysis Explanation

Priority: Medium.

Stimulus / Response Sequence

1. Prediction is generated.
2. System displays the analysis pipeline.
3. System explains BERT representation.
4. System explains semantic relationships.
5. System explains GCN classification.

Functional Requirements

ID	Requirement	Priority
REQ-32	The system shall explain the BERT representation stage.	Medium
REQ-33	The system shall explain the semantic graph stage.	Medium

ID	Requirement	Priority
REQ-34	The system shall explain the GCN classification stage.	Medium
REQ-35	The system shall display the overall system architecture.	Medium

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Requirement	Priority
NFR-1	The system should provide prediction results within a reasonable time under the available hardware configuration.	Medium
NFR-2	The system shall load the BERT and GCN models only when required and may cache them during application execution.	Medium
NFR-3	The system shall use inference mode when generating predictions to avoid unnecessary gradient computation.	High
NFR-4	The system should efficiently process existing article embeddings when calculating semantic similarity.	Medium
NFR-5	The interface shall provide a visible processing indicator while prediction is being performed.	Medium

5.2 Safety Requirements

The system is an informational machine-learning prototype and does not directly control physical devices.

ID	Requirement	Priority
SAFE-1	The system shall not present an automated prediction as definitive proof that a news article is factually true or false.	High
SAFE-2	The system shall communicate that predictions are model-generated classifications.	High
SAFE-3	Users should independently verify important information through reliable sources before acting on it.	Medium

5.3 Security Requirements

ID	Requirement	Priority
SEC-1	The prototype shall not require users to provide authentication information.	Medium
SEC-2	User-submitted article text shall only be processed for the requested analysis.	High
SEC-3	The application shall not intentionally expose internal model files through the user interface.	High
SEC-4	Model and dataset files shall be stored in controlled project directories.	High
SEC-5	The system shall validate user input before processing it.	High
SEC-6	If deployed publicly, appropriate application and server security mechanisms shall be implemented.	Medium

5.4 Software Quality Attributes

Attribute	Requirement
Usability	QA-1: Submit an article without technical knowledge. QA-2: Make prediction results visually prominent and easy to understand.
Reliability	QA-3: Handle empty or invalid input without terminating the application.
Maintainability	QA-4: Keep model loading, prediction, graph construction, and UI functionality logically separated.
Portability	QA-5: Operate on supported Windows, Linux, and macOS environments with required dependencies.
Testability	QA-6: Structure the prediction pipeline so individual components can be tested independently.
Scalability	QA-7: Permit future expansion to larger news datasets.
Interpretability	QA-8: Provide related articles and an analysis pipeline to help users understand the computational process.

5.5 Business Rules

ID	Requirement	Priority
BR-1	Only submitted news text shall be analyzed by the prediction interface.	High

ID	Requirement	Priority
BR-2	The system shall classify news into the two supported classes: REAL and FAKE.	High
BR-3	The prediction shall be generated using the trained machine-learning model.	High
BR-4	Similar articles shall be selected based on semantic similarity.	Medium
BR-5	The system shall not treat a machine-learning classification as a substitute for human fact verification.	High

6. Other Requirements

6.1 Dataset Requirements

The system shall use a labeled news dataset containing news articles and corresponding REAL/FAKE labels. The current prototype dataset contains approximately 6,335 news articles with two classes: REAL and FAKE.

- Processed fields: **content** and **label**.
- Numeric label mapping: **REAL** → **0**; **FAKE** → **1**.

6.2 Model Requirements

The system shall use BERT for contextual text representation, cosine similarity for semantic relationship identification, and a Graph Convolutional Network for graph-based classification.

Current GCN Architecture

Stage	Configuration
Input	768-dimensional BERT representation
Layer 1	GCNConv: 768 → 128
Activation	ReLU
Layer 2	GCNConv: 128 → 64
Activation	ReLU
Layer 3	GCNConv: 64 → 2
Output	REAL / FAKE

6.3 Storage Requirements

The application shall maintain the following major project resources:

```
models/
■■■ bert-base-uncased/
■■■ gnn_model.pt

data/
■■■ raw/
■ ■■■ fake_news.csv
■■■ processed/
■■■ clean_news.csv
■■■ news_graph.pt
```

The trained model and processed graph shall be available before prediction functionality is executed.

6.4 Deployment Requirements

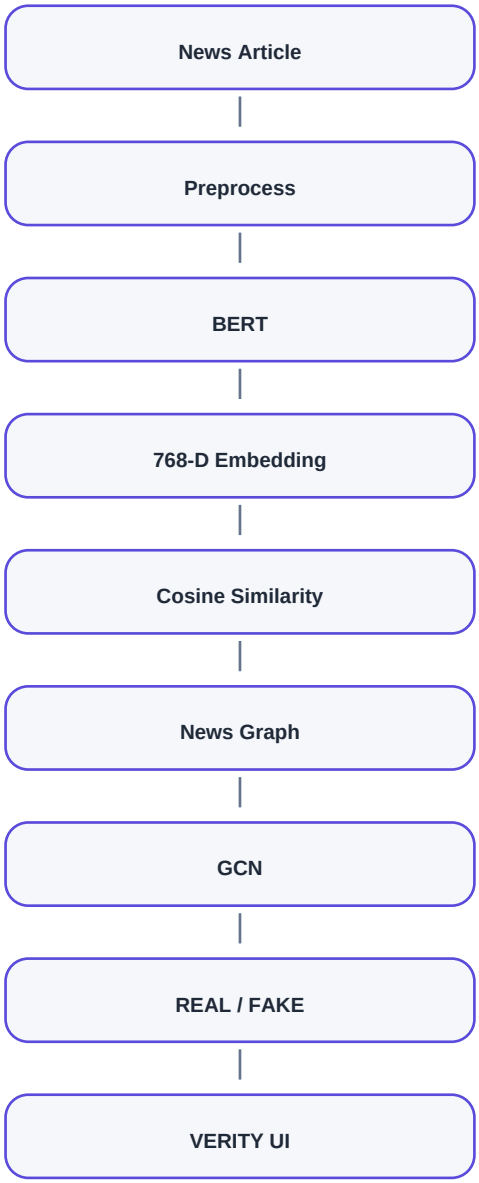
- The prototype shall be executable using Streamlit.
- The application may be started using: **python -m streamlit run app.py**.
- The application shall be accessible through a supported web browser.

Appendix A: Glossary

Term	Definition
AI	Artificial Intelligence.
BERT	Bidirectional Encoder Representations from Transformers.
Classification	Assigning an input to a predefined class.
Cosine Similarity	A measure of similarity between two vectors.
Embedding	Numerical representation of text or other data.
FAKE	Fake-news classification label.
GCN	Graph Convolutional Network.
GNN	Graph Neural Network.
Graph	A structure consisting of nodes and edges.
NLP	Natural Language Processing.
Node	An individual article represented in the news graph.
Edge	A relationship between two graph nodes.
REAL	Real-news classification label.
Softmax	Function used to convert model scores into class probabilities.
Tokenization	Conversion of text into tokens suitable for model processing.
Transformer	Neural-network architecture used extensively in NLP.
Inference	Using a trained model to generate predictions.

Appendix B: Analysis Models

B.1 Level-0 Data Flow



B.2 Graph Representation

Each news article is represented as a node. Semantic similarity determines relationships between nodes. The submitted article is added as a new node and connected to its five most semantically similar existing articles.

Appendix C: To Be Determined List

The following items may be refined during further development:

ID	Requirement	Priority
TBD-1	Final deployment environment.	—
TBD-2	Final model performance metrics after completion of GCN training and evaluation.	—
TBD-3	Model retraining frequency for future versions.	—
TBD-4	Maximum supported dataset size for production deployment.	—
TBD-5	Additional languages to be supported beyond the current BERT configuration.	—
TBD-6	Additional fact-checking or external news-source integrations for future versions.	—

End of Software Requirements Specification — VERITY — Version 1.0